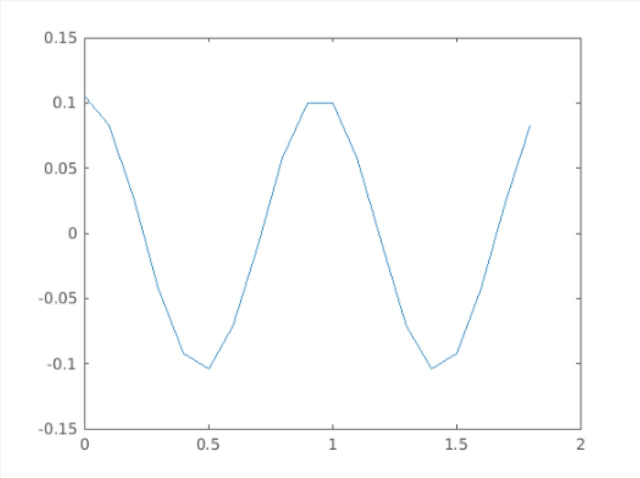
myifft

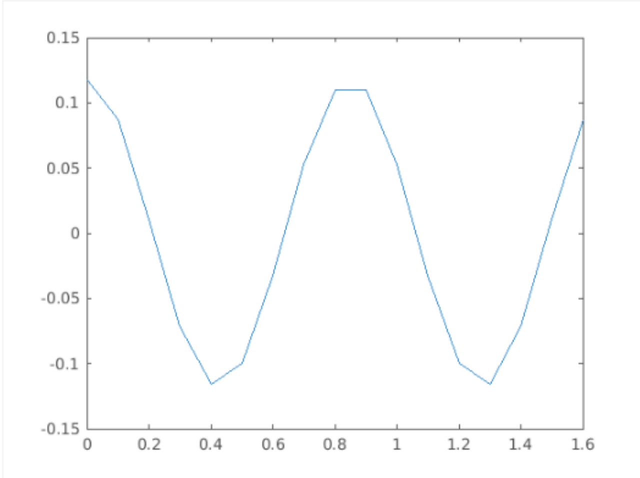
Write your own 'myifft' function identically to the one implemented in the lecture. It takes as input a [Nx1] Fourier transform vector **yfft**, [N
frequency **Fs**. It outputs **y**, the time domain signal corresponding to the input Fourier transform vector.

Using it in the example code, you should see exactly this output:

Output

```
yEven =

  Columns 1 through 17

    0.1053    0.0831    0.0258   -0.0423   -0.0926   -0.1038   -0.0713   -0.0087    0.0576    0.0996    0.0996    0.0576   -0.0087

  Columns 18 through 19

    0.0258    0.0831


yOdd =

    0.1176    0.0869    0.0109   -0.0709   -0.1156   -0.1000   -0.0322    0.0524    0.1097    0.1097    0.0524   -0.0322   -0.1000
```





Function

Save Reset MATLAB Documentation (https://www.mathworks.com/help/)

```
function y = myifft(yfft, f, Fs)
% Replace redundant part of the fft that we trimmed away:
%   if the last frequency in f equals Fs/2,
    if f(end) == Fs/2
%       then we need to append the conjugate of the mirrored entries in yfft from 2 to length of yfft minus
        y = [yfft; conj(flipud(yfft(2:end-1)))];

%   else
    else
%       then we need to append the conjugate of the mirrored entries in yfft from 2 to length of yfft
        y = [yfft; conj(flipud(yfft(2:end)))];
    end

% perform ifft
    y = ifft(y);
end
```

Code to call your function

Reset

```
% Even length y
Fs = 10;
yfft = [0 0 1 0 0 0 0 0 0 0]'; % magnitude spectrum with single frequency corresponds to a single cosine
f = 0:5;
y = myifft(yfft,Fs,f);
t = 0:1/Fs:(length(y)-1)/Fs;
plot(t,y)
yEven = y' % = [0.1053    0.0831    0.0258   -0.0423   -0.0926 ...]

% Odd length y
Fs = 10;
yfft = [0 0 1 0 0 0 0 0 0]';
f = 0:10/9:40/9;
y = myifft(yfft,Fs,f);
t = 0:1/Fs:(length(y)-1)/Fs;
```

▶ Run Function

Assessment: All Tests Passed

Submit

✔ **Is output correct for even length random-valued outputs?**

✔ **Is output correct for odd length random-valued outputs?**